有道翻译 支持免费文档翻译60万字符/年 立即下载→ https://fanyi.youdao.com/download

如何：在 Python 中从图像中提取表（OpenCV 和 OCR）

2023 年 4 月 17 日发布，2023 年 5 月 4 日更新，作者：Khoj Badami

在本指南中，我们将使用 OpenCV 和 TesseractOCR 在 Python 中从图像中提取表。我们将使用一盒巧克力背面的营养标签图像。我们将假设你正在做一个**项目，其中这些类型的营养表需要数字化。**

注意：如果您**尝试按照您的**情况使用此代码，您可能会发现它不能**很好地工作。**这是因为有很多小**的事情你可以**根据你的情况来调整和完善。在你理解了代码之后，你将处于一个非常好的位置来做这件事。在指南的最后，我还建议了一些你可以采取的进一步改进措施。

最终产品的视频&过程的视觉概述

看看这个视频，它应该**可以帮助你看到整个过程的运作。**

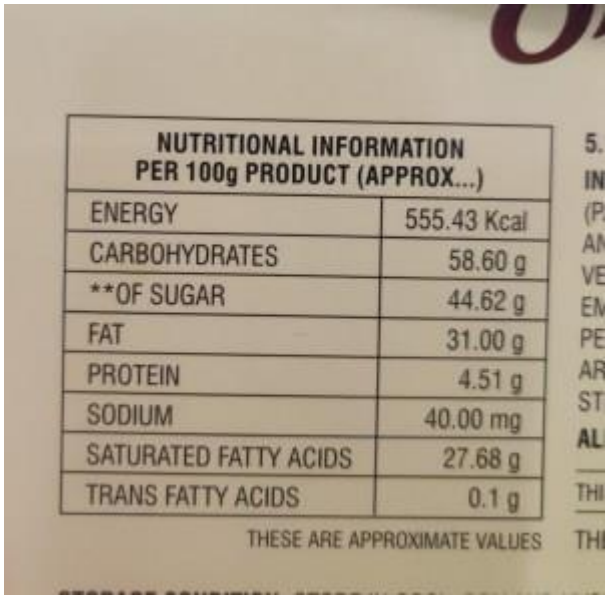
的代码

代码包含 **3 个类，用于过程的 3 个阶段**。我在 GitHub 上创建了一个公共仓库，并将代码上传到那里。

阶段 A：检测桌子

阶段 A 就是**看图像，找到最可能是桌子的区域。**这是通过简单地**寻找图像中最大的“盒子”**来完成的。

在我们继续之前，让我们先看看阶段 A 的输入和输出：



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES

上图为**原始图像**。

阶段 A 后的输出如下：

NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

那么，让我们开始看看我们是如何实现上述目标的。

阶段 A：简化总体规划

基本上，我们要**把图像拍下来，把所有的线条都加粗**。一旦我们这样做了，我们就找到了所有的“轮廓”。以下是官方 OpenCV 文档对“轮廓”的解释：

> **什么是轮廓？**

> 轮廓可以简单地解释为**连接所有连续点（沿着边界）的曲线，具有相同的颜色或强度。**轮廓是形状分析和物体检测和识别的有用工具。

一旦我们有了所有的轮廓，我们就会**找到最大的矩形**，并希望这就是我们的表格。

我们首先“加粗”线条的原因是，我们试图确保我们的桌子外部有一个强大而清晰的轮廓。如果我们不这样做，内部单元格框可能会与外部框混淆。（你可以尝试消除这个增厚步骤，看看它是否适用于你的用例。）

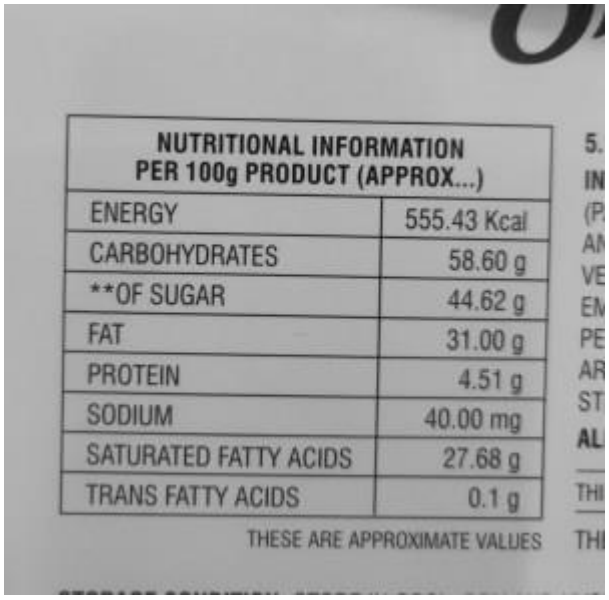
最后，**一旦我们有了最大的矩形**，我们将使用 opencv 内置函数来**纠正透视图**。

步骤 A1：预处理

这一步由 **4 个子步骤**组成。下面我将列出这些步骤，以及我为什么要这样做，还有官方文档的链接，这样如果你需要的话，你可以更深入地了解这些主题。

一步不	名字	我们为什么要这样做？	链接到官方文档
1	Grey-scaling	我们不关心颜色信息。去掉颜色信息会让处理速度更快。	链接
2	阈值	我们甚至不关心灰色的深浅。所以，我们将图像简化为“黑色”或“白色”像素。这也使事情变得更快，并为进一步的操作所需要。	链接
3	反相	图像是这样的，文字是黑色的，背景是白色的。我们需要将其反转，这样我们才能应用下一个操作。	链接
4	扩张	在这里，我们将使图像中的所有线条和任何形状变粗。这将帮助我们正确识别“轮廓”，并希望“轮廓”构成最大的框。我们希望最大的盒子是桌子。	链接

下面是图片在经历上述步骤后的变化：

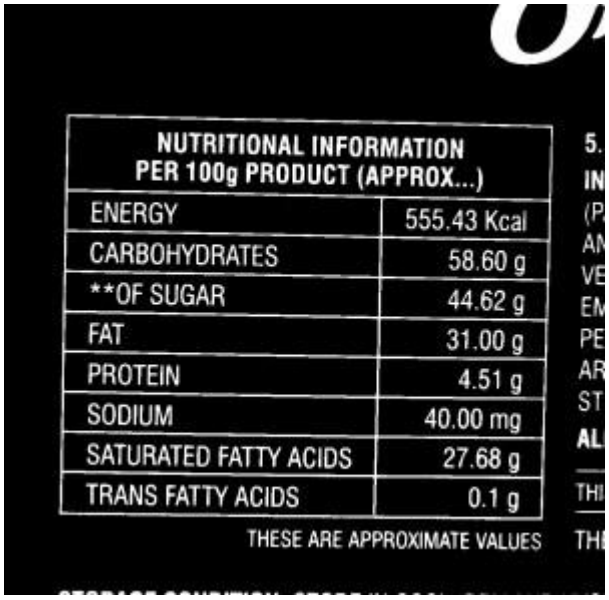


NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES

NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES





下面是**完成上述转换的 Python 代码。**函数的名称应该能清楚地说明发生了什么。为了理解一些参数等，你需要查看上表中提供的官方文档链接：

```
def convert_image_to_grayscale(自我):
自我。grayscale_image = cv2.cvtColor（self.）形象,cv2.COLOR_BGR2GRAY)
def threshold_image(自我):
自我。Thresholded_image = cv2.threshold（self.）Grayscale_image, 0,255,
cv2.。THRESH_BINARY + cv2.THRESH_OTSU)[1]
def invert_image(自我):
自我。Inverted_image = cv2.bitwise_not（self.thresholded_image）
def dilate_image(自我):
自我。Dilated_image = cv2.dilate（self.）inverted_image, None,
iterations=5)
```

步骤 A2：查找并过滤轮廓以找到最大的矩形轮廓

现在我们要做以下事情：

1. 让 OpenCV **找到所有的轮廓。**
2. 循环遍历这些轮廓，**只留下“矩形”轮廓。**
3. **按面积找到最大的等高线**（希望这个有我们的表格）

当我们完成上面的步骤后，它的视觉效果是这样的：

注意：即使我们在上面生成的预处理图像上做寻找轮廓等工作，**轮廓的打印也是在原始图像上完成的。**这样做是因为我们不能在二进制黑白图像上绘制“绿色”线。没有办法用黑白来表示绿色。



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

THESE ARE APPROXIMATE VALUES

下面是这一部分的代码：

```
def find_contours(自我):
自我。轮廓,自我。hierarchy = cv2.findContours（self.）dilated_image cv2。
RETR_TREE cv2.CHAIN_APPROX_SIMPLE)
#下面的线条被添加以显示所有轮廓
#这不是必需的，但它对调试很有用
自我。Image_with_all_contours = self.image.copy（）
cv2.drawContours(自我。image_with_all_contours,自我。Contours, -1,
(0,255, 0), 3)
def filter_contours_and_leave_only_rectangles(自我):
自我。Rectangular_contours = []
对于 self.contours 中的轮廓：
Peri = cv2。arcLength(轮廓,真的)
Approx = cv2。approxPolyDP（contour, 0.02 * peri, True）
如果 len(approx) == 4：
self.rectangular_contours.append(大约)
添加下面的线条以显示所有矩形轮廓
#这不是必需的，但它对调试很有用
自我。Image_with_only_rectangular_contours = self.image.copy（）
cv2.drawContours(self。image_with_only_rectangular_contours,自我。
Rectangular_contours, -1, (0,255,0), 3)
def find_largest_contour_by_area(自我):
Max_area = 0
自我。contour_with_max_area = None
对于 self.rectangular_contours 中的轮廓：
area = cv2.contourArea（轮廓）
如果 area > max_area：
Max_area = area
自我。Contour_with_max_area = contour
#下面的线条被添加以显示具有最大面积的轮廓
#这不是必需的，但它对调试很有用
自我。Image_with_contour_with_max_area = self.image.copy（）
cv2.drawContours(self。image_with_contour_with_max_area,自我。
Contour_with_max_area], -1, (0,255,0), 3)
```

注意，在上面的所有函数中，**最后几行都是关于显示图像的轮廓的。**这并不是严格需要的。但是，无论如何，它是给定的，以便您**可以调试，如果有些东西不像您期望的那样工作。**

正如你在上面的第一个函数中看到的，我们正在做所有的工作来找到“放大”图像的轮廓。当显示轮廓时，我们是在原始图像的副本上绘制它们。

下面是一些**官方文档的链接**，你可能需要这些链接来理解上面的代码：

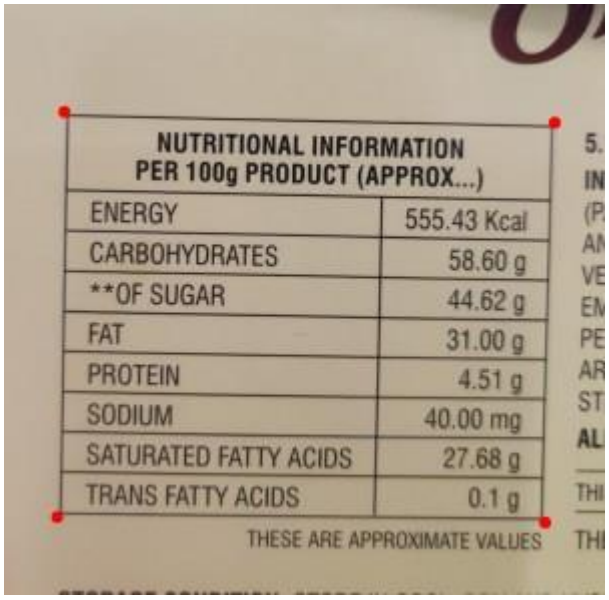
- Contours: Getting Started
- 使用 approxPolyDP 近似轮廓
- 找出轮廓的面积和其他细节：轮廓特征

步骤 A3：透视校正

如上图所示，我们**有了最大的矩形。**现在我们**只需要隔离图像的那一部分，还需要固定透视。**为了做到这一点，我们需要给 OpenCV 所有的信息，关于旧图像上的点的坐标是什么，以及它们需要如何映射到新的转换后的图像。为了做到这一点，我们将需要做以下工作：

1. **将**我们在上面找到的最大矩形的轮廓上的 4 个点排序。
2. 根据这些点，以及矩形的宽度和高度，**估计要生成的新图像的大小**
3. 最后，**应用透视变换**

大部分的工作只是代码和逻辑。所以在图像方面并没有太多要展示的东西。但下面我在原图上**画了 4 个点，它们构成了矩形的各个角**。下面的代码展示了如何做到这一点。这将有助于调试。



让我们看一下代码。变量和方法的命名应该能让我们很清楚发生了什么：

```
def order_points_in_the_contour_with_max_area(自我):
自我。Contour_with_max_area_ordered = self.order_points
（self.contour_with_max_area）
#下面的代码是用来绘制图像上的点
#这不是透视变换所必需的
#它将帮助你理解和调试代码
自我。image_with_points_plot = self.image.copy（）
```

对于 self.contour_with_max_area_ordered 中的点：
Point_coordinates = (int（point[0]), int(point[1])）
自我。image_with_points_plot = cv2.circle(self。image_with_points_plot, point_coordinates, 10, (0,0,255), -1)
def calculate_new_width_and_height_of_image(自我):
Existing_image_width = self.image.shape[1] .
Existing_image_width_reduced_by_10_percent = int（existing_image_width * 0.9）
distance_between_top_left_and_top_right = self.
calculatedistancebetween2points （self.
calculatedistancebetween2points）contour_with_max_area_ordered [0], self.contour_with_max_area_ordered [1])
distance_between_top_left_and_bottom_left = self.
calculatedistancebetween2points （self.
calculatedistancebetween2points）contour_with_max_area_ordered [0], self.contour_with_max_area_ordered [3])
Aspect_ratio = distance_between_top_left_and_bottom_left / distance_between_top_left_and_top_right
自我。New_image_width = existing_image_width_reduced_by_10_percent
自我。New_image_height = int(self。New_image_width * aspect_ratio)
def apply_perspective_transform(自我):
Pts1 = np.float32（self.contour_with_max_area_ordered）
Pts2 = np.; Float32 ([[0,0], [self.]New_image_width, 0], [self.]new_image_width,自我。New_image_height], [0, self.new_image_height]])
Matrix = cv2。getPerspectiveTransform (pts1 pts2)
自我。perspective_corrected_image = cv2.warpPerspective（self.;）Image, matrix, （self.）new_image_width, self.new_image_height))
#下面是辅助函数
defcalculatedistancebetween2points (self, p1, p2):
说= ((p2 [0] - p1 [0]) * * 2 + (p2 [1] - [1]) * * 2) 0.5 * *
回来说
Def order_points(self, pts):
initialzie 将排序的坐标列表
#使列表中的第一个条目位于左上角，
#第二个条目在右上方，第三个是
bottom-right，第四个是 bottom-left
PTS = PTS。重塑(4,2)
Rect = np。零点（(4,2),dtype=“float32”）
#左上角的点会有最小的和，而
#右下的点将有最大的和
S = pts.sum（axis=1）
Rect [0] = pts[np.argmin(s)]
Rect [2] = pts[np.argmax(s)]

```
#现在，计算点之间的差值
#右上点的差异最小，
#而左下角的点会有最大的差异
Diff = np。diff(分轴= 1)
Rect [1] = pts[np.argmin(diff)]
Rect [3] = pts[np.argmax(diff)]
#返回有序坐标
返回矩形
```

下面是[关于透视变换的官方文档链接。]

在透视变换之后，我们得到了一张这样的图片：

NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

我们要做的最后一件事是**给图像添加一些填充**。这将**需要在我们移除线条的下一个阶段**。如果没有这个，我注意到线条不会被完全移除。

下面是添加填充的代码：

```
def add_10_percent_padding(自我):
Image_height = self.image.shape[0] .
Padding = int（image_height * 0.1）
自我。perspective_corrected_image_with_padding = cv2.copyMakeBorder
（self.;）Perspective_corrected_image, padding, padding, padding,
padding, cv2。BORDER_CONSTANT, value=[255,255,255])
```

以下是所使用方法的官方文档：为[图像添加边框]

当我们完成这些后，我们得到一个像这样的图像：

NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

阶段 B：去掉线条

这个阶段主要是**去除桌子上的线条**。这将有助于我们对 **OCR 过程有一个清晰的图像**。最后，图像剩下的唯一东西将是表格单元格中的文本。

让我们来看看这一阶段**结束时的最终结果会是什么**样子：



步骤 B1：预处理

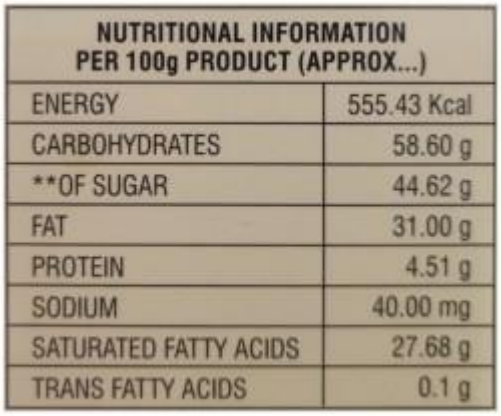
这与**我们在上一阶段所做的预处理非常相似**。

注意：你甚至可以选择跳过这一步，使用上一阶段生成的图像。我再次这样做的原因是为了清晰，也是为了让构成这个阶段的 Python 类可以被模块化地使用，给定一个只包含一个表而没有其他内容的图像。

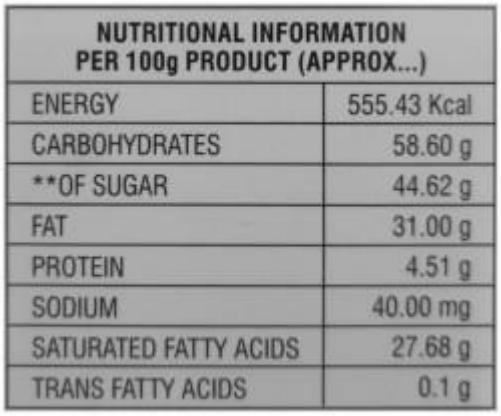
这个阶段从上一个阶段**获取一个全彩色图像**以及填充，并**将其转换为一个倒置的二进制图像。**

一步不	名字	我们为什么要这样做？	链接到官方文档
1	Grey-scaling	我们不关心颜色信息。去掉颜色信息会让处理速度更快。	链接
2	阈值	我们甚至不关心灰色的阴影。所以，我们将图像简化为“黑色”或“白色”像素。这也使事情变得更快，并为进一步的操作所需要。	链接
3	反相	图像是这样的，文字是黑色的，背景是白色的。我们需要将其反转，这样我们才能应用下一个操作。	链接

下面是图像在经历这些阶段时的变化：



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g



NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g

NUTRITIONAL INFORMATION PER 100g PRODUCT (APPROX...)	
ENERGY	555.43 Kcal
CARBOHYDRATES	58.60 g
**OF SUGAR	44.62 g
FAT	31.00 g
PROTEIN	4.51 g
SODIUM	40.00 mg
SATURATED FATTY ACIDS	27.68 g
TRANS FATTY ACIDS	0.1 g



下面是代码：

```
def grayscale_image(自我):
自我。grey = cv2.cvtColor（self.）形象,cv2.COLOR_BGR2GRAY)
def threshold_image(自我):
自我。Thresholded_image = cv2.threshold（self.）grey, 127,255,
cv2.THRESH_BINARY)[1] .
def invert_image(自我):
自我。Inverted_image = cv2.bitwise_not（self.thresholded_image）
```

步骤 B2：侵蚀垂直线

为了理解竖线和所有文字是如何被侵蚀掉的，你必须正确理解“侵蚀”和“膨胀”的概念。

下面，我从 YouTube 上收集了一些关于这个话题的最佳视频。看完这些，你会对正在发生的事情有更好的感觉：

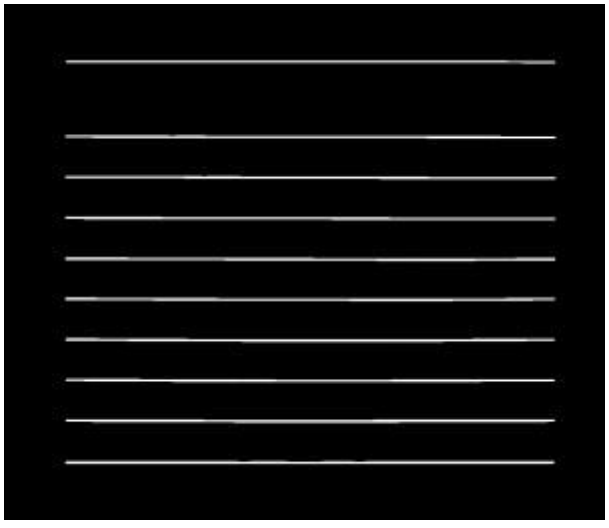
现在，看了上面的内容，在侵蚀和膨胀的背景下，**应该相当清楚什么是“内核”**了。基本上，它**是一种接管图像的形状**，用于通过在原始**图像上移除或添加像素来变换底层**图像。

现在，有了这些，下面用于删除所有垂直线条的代码应该有一些意义：

```
def erode_vertical_lines(自我):
Hor = np.array（[[1,1,1,1,1,1,1]]）
自我。Vertical_lines_eroded_image = cv2. erosion （self. self.）
Inverted_image, hor, iterations=10)
自我。Vertical_lines_eroded_image = cv2.dilate（self.）
Vertical_lines_eroded_image, hor, iterations=10)
```

如上图所示，我们正在**创建一个“内核”，它只是一条水平线。**然后，我们正在**侵蚀所有与内核匹配的东西。**一旦这一步完成，我们就在**扩张剩下的东西。**我们之所以做**第二步，是因为在侵蚀步骤之后，表格的线条变得有点太短**了。所以，我们要把它们带回来。

在这个过程的最后，图像看起来像这样：



这个过程看起来可能**有点太神奇了。**所以，我建议**你玩**一下不同的代码行，尝试不同的东西，比如：

1. 当你放入不同的内核时，会发生什么

2. 侵蚀后不膨胀会怎么样

以下是关于侵蚀和膨胀的官方 OpenCV 文档。

他们也有一个关于这个主题的很好的教程，叫做：通过使用形态学操作提取水平线和垂直线

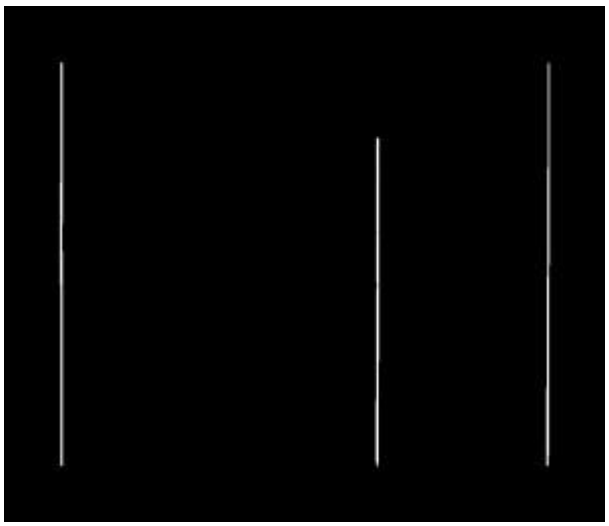
步骤 B3：侵蚀水平线

接下来，我们将使用**类似的过程来侵蚀掉水平线。**解释和上面一样，所以我就不再赘述了。

下面是代码：

```
def erode_horizontal_lines(自我):
Ver = np.array([[1],
[1],
[1],
[1],
[1],
[1],
[1]])
自我。Horizontal_lines_eroded_image = cv2. erosion （self.）
Inverted_image, ver, iterations=10)
自我。Horizontal_lines_eroded_image = cv2.dilate（self.）
Horizontal_lines_eroded_image, ver, iterations=10)
```

上面的代码运行完之后，我们得到了下面这张图片：

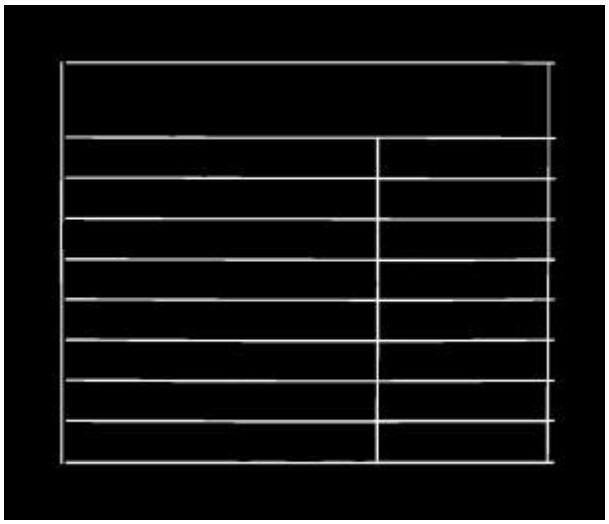


步骤 B4：组合垂直线和水平线

现在，我们将使用简单的“**add**”操作将水平线和垂直线组合起来。它只是**将两张图片中的白色像素**相加：

```
def combine_eroded_images(自我):
self.combined_image = cv2.add（self.）vertical_lines_eroded_image
self.horizontal_lines_eroded_image)
```

上面的结果是：



接下来，我们将**再次使用“dilate”来“thickened”这些线条。**

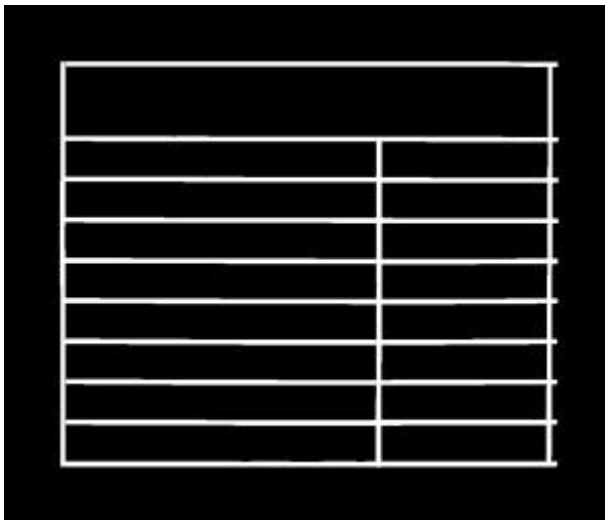
我们为什么要增厚？因为**就像我们在上面做“加”一样，我们也可以做“减”，去掉任何被“白色”覆盖的区域**。但是，在我们这样做之前，我们要使线条好看而粗，这样它们**就可以很容易地覆盖底层的线条。**

所以，为了使线条变粗，我们使用下面的代码：

```
def dilate_combined_image_to_make_lines_thicker(自我):
kernel = cv2. getstructuringelement （cv2.）Morph_rect, (2,2))
Self.combined_image_dilated = cv2.dilate（self.combined_image, kernel,
iterations=5）
```

在这里你可以看到我们正在**使用** getStructuringElement **来创建一个漂亮的简单矩形内核。**简单的内核将**遍历图像并使其变厚。**类似于上面你可能看过的视频。

最后，我们得到的图像是这样的：



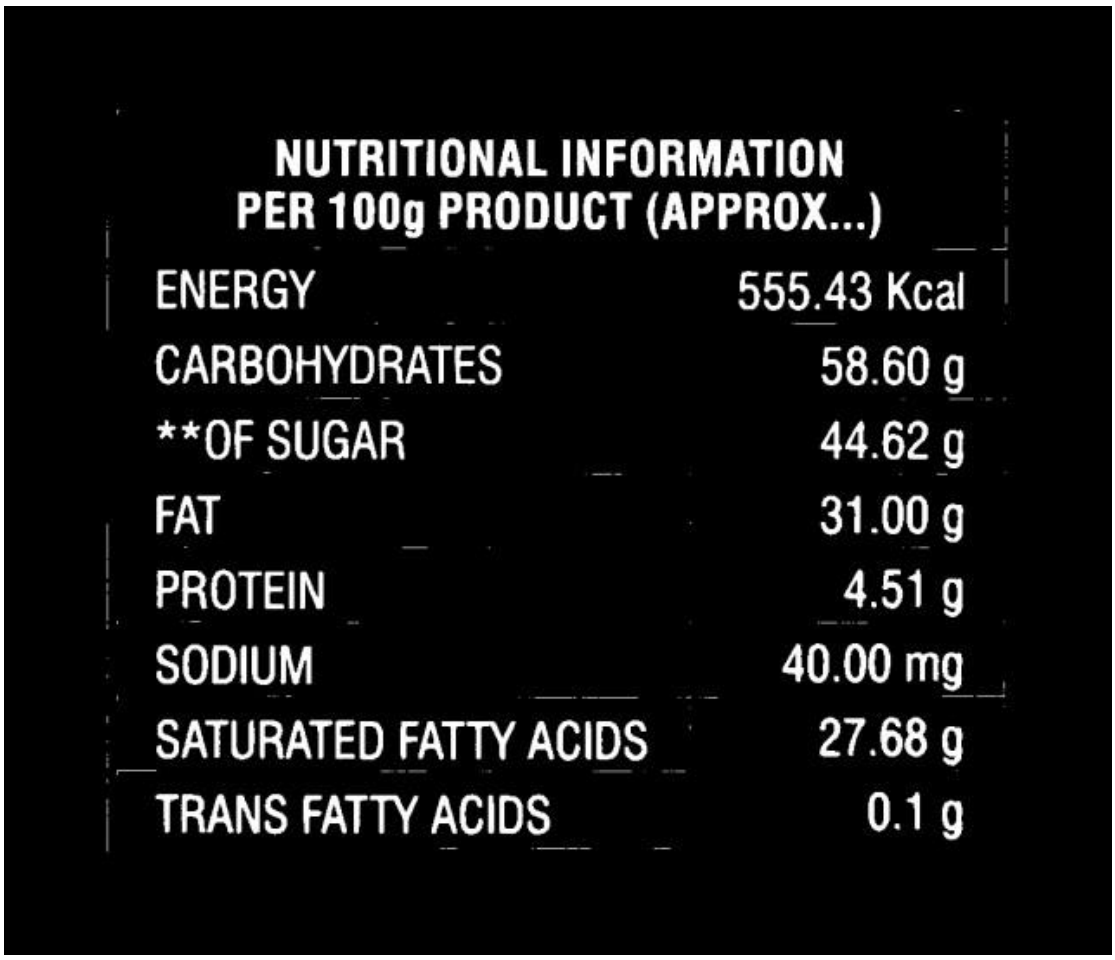
步骤 B5：移除线条

这部分很简单。现在我们有了一个**只由表格的线条组成的图像**，我们可以做一个**“减法”，得到一个没有线条的图像。**

这样做的代码是：

```
def subtract_combined_and_dilated_image_from_original_image(自我):
自我。Image_without_lines = cv2.subtract（self.）inverted_image
self.combined_image_dilated)
```

有了这个，我们得到了一个像这样的图像：



我在**上面放了一张比平时大的图片，这样你就能注意到问题了。**线条消失了（某种程度上），但**这里和那里有这些小的线条碎片。**我们也可以通过去除杂音来消除这些碎片：

```
def remove_noise_with_erode_and_dilate(自我):
kernel = cv2. getstructuringelement （cv2.）Morph_rect, (2,2))
自我。Image_without_lines_noise_removed = cv2. erosion （self. self.）
Image_without_lines, kernel, iterations=1)
自我。Image_without_lines_noise_removed = cv2.dilate(self。
Image_without_lines_noise_removed, kernel, iterations=1)
```

这里**我们将再次使用 erosion 和 dilate。**我们使用的是和上面一样的简单内核。净效果是，所有的 1,2 像素厚的小比特都被完全侵蚀掉了。较厚的区域保留了下来。一旦它们被侵蚀掉，我们就可以扩大和加厚文本和其他被侵蚀掉的区域。

最终的效果是一个干净的图像。所有的小杂音都消失了。只剩下文字。我们已经完成了这个阶段，并为最后的 OCR 阶段做好了准备：



阶段 C：查找单元格并使用 OCR 提取文本

阶段 C：高级计划

以下是高层规划：

1. 现在**我们有了一张只有文字的图片**，我们要**把所有的文字都转换成 blobs。**（像往常一样使用扩张）。
2. 然后我们将**使用 findContours**（我们在上面看到过）来**找到所有 blobs 的位置。**
3. 然后我们将在 **blobs 周围绘制边界框。**
4. 然后我们将**图像分割成只包含单词的小框。**
5. 在这种情况下，我们将**把每个图像切片发送到 OCR 工具**（我们将使用自由 **Tesseract OCR**）。我们将得到单词的文本版本。
6. 最后，我们**用一点逻辑来计算表格的行和列**，我们用 CSV 来构建整个东西

下面我们来看一下如何执行上面的计划。

步骤 C1：使用扩张将单词转换成 Blobs

到这里，（因为我们已经看到了很多扩张），这一步发生的事情应该很清楚了：

首先，让我们看一下代码：

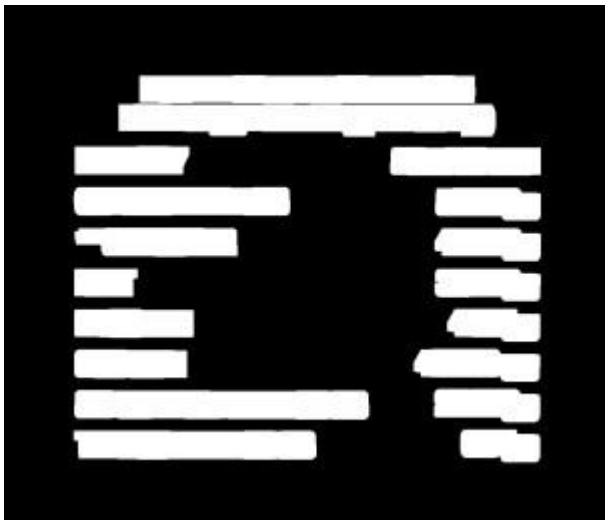
```
def dilate_image(自我):
Kernel_to_remove_gaps_between_words = np.array([
(1, 1, 1, 1, 1, 1, 1, 1, 1, 1),
(1, 1, 1, 1, 1, 1, 1, 1, 1, 1)
])
```

```
自我。Dilated_image = cv2.dilate（self.）Thresholded_image,
kernel_to_remove_gaps_between_words, iterations=5)
Simple_kernel = np.ones（(5,5), np.uint8）
自我。Dilated_image = cv2.dilate（self.）Dilated_image, simple_kernel,
iterations=2)
```

如你所见，我们使用了一个**长水平内核。**这有助于我们**扩大单词，把它们变成水平的污迹。**（请看下图）。

然后我们也用了**平方核只是为了填补空白。**

最终的结果是这样的：



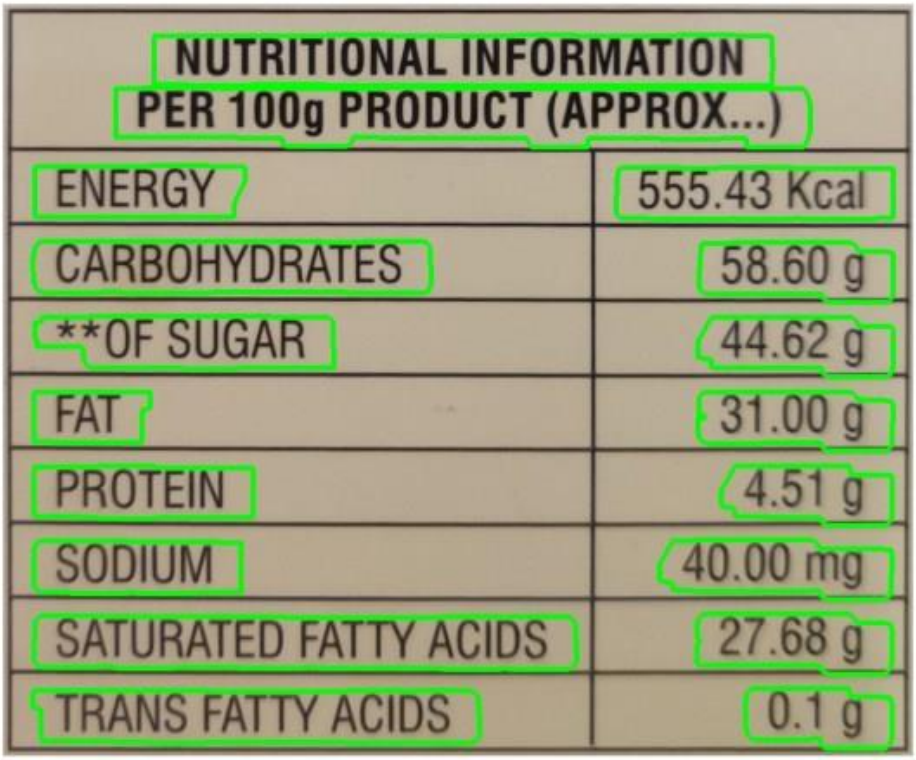
步骤 C2：找到斑点的轮廓

接下来，我们需要**使用** findContours **方法找到所有这些污迹。**这也是我们之前在第一阶段做过的事情。

下面是**查找轮廓线并在原始图像上绘制轮廓线**的代码，以达到可视化的目的。

```
def find_contours(自我):
result = cv2.findContours（self.）dilated_image cv2。RETR_TREE
cv2.CHAIN_APPROX_SIMPLE)
自我。Contours = result[0]
#下面的代码仅用于可视化目的。
# OCR 没有必要工作。
自我。Image_with_contours_drawn = self.original_image.copy（）
cv2.drawContours(自我。image_with_contours_drawn,自我。Contours, -1,
(0,255, 0), 3)
```

在这之后，我们将得到一个像这样的图像：



步骤 C3：将斑点转换为边界框

接下来，我们将**使用一个新的 OpenCV 函数：boundingRect**（在官方文档的本页搜索该函数）

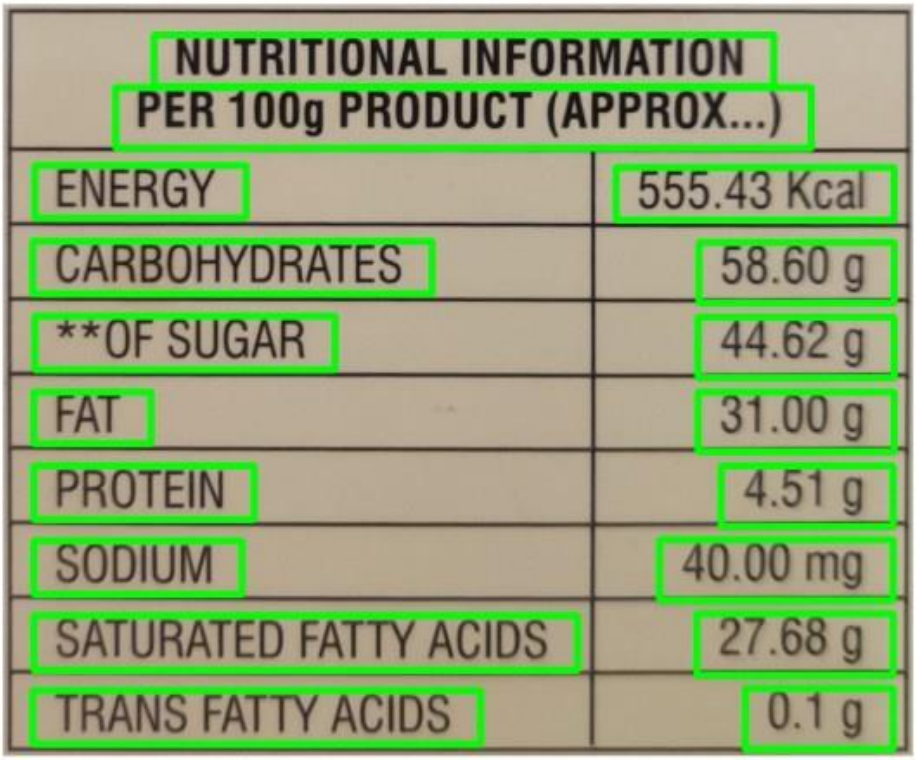
该函数**接受轮廓**（由许多点组成），并**将其简化为一个可以完全包围轮廓形状的框。**

下面是代码：

```
def convert_contours_to_bounding_boxes(自我):
自我。Bounding_boxes = []
自我。Image_with_all_bounding_boxes = self.original_image.copy（）
对于 self.contours 中的轮廓：
x, y, w, h = cv2.boundingRect（contour）
self.bounding_boxes。Append （(x, y, w, h)）
#下面这行是关于
在图片上画一个矩形，形状为
```

#边界框。OCR 不需要它。
#它只是为了调试而添加的。
自我。Image_with_all_bounding_boxes = cv2.rectangle（self.）
image_with_all_bounding_boxes (x, y), (x + w, y + h) (0 255 0), 5)

这就是图片中的东西：



步骤 C4：按 X 和 Y 坐标排序边界框以制作行和列

这一步是一些很好的**老式逻辑。不需要 OpenCV**。我们只需要**创建一个包围框数组的数组来表示表格的行和列。**

下面是代码：

```
def get_mean_height_of_bounding_boxes(自我):
Heights = []
对于 self.bounding_boxes 中的 bounding_box：
X, y, w, h = bounding_box
heights.append (h)
返回 np.mean(高度)
```

```
def sort_bounding_boxes_by_y_coordinate(自我):
自我。Bounding_boxes = sorted（self.）Bounding_boxes, key=lambda x: x[1])
def club_all_bounding_boxes_by_similar_y_coordinates_into_rows(自我):
自我。Rows = []
Half_of_mean_height = self。Mean_height / 2
Current_row = [self.]bounding_boxes [0]]
对于 self.bounding_boxes[1:]中的 bounding_box：
Current_bounding_box_y = bounding_box[1]
Previous_bounding_box_y = current_row[-1][1]
Distance_between_bounding_boxes = abs（current_bounding_box_y -
previous_bounding_box_y）
如果 distance_between_bounding_boxes <= half_of_mean_height：
current_row.append (bounding_box)
其他:
self.rows.append (current_row)
Current_row = [bounding_box]
self.rows.append (current_row)
def sort_all_rows_by_x_coordinate(自我):
对于 self.rows 中的 row：
行。Sort （key=lambda x: x[0]）
```

让我试着用英语解释一下这是怎么回事，这样有助于理解代码：

1. **找出盒子的平均高度。**我们这样做是因为我们想用平均高度来决定一个盒子是在这一行还是下一行。如果我们发现 y 变化很大，那么我们处理的是下一行的一个方框。
2. 接下来，我们根据 **y 坐标对盒子进行排序。**这将有助于**确保同一行中的所有盒子都在一起。**
3. 接下来，我们开始**制作“行”数组。**我们通过**查看 Y 坐标来做到这一点。**如果它与上**一个有很大的变化，我们就在一个新的行。**如果它有**一点变化，我们将这个盒子添加到同一行。**
4. 最后，我们得到一个**数组，其中有代表行的子数组。**
5. 最后，我们**按 X** 坐标对构成行的所有边界框进行排序。这确保了它们在**行内的顺序都是正确的。**

步骤 C5：使用 OCR 从边界框中提取文本

现在，我们循环遍历所有行，**并开始基于边界框制作小图像切片。**每个切片都有一个单词。我们保存图像，然后对其运行 TesseractOCR。

为了让它工作，你**需要首先安装 Tesseract。**如何做到这一点的细节可以在这里找到。

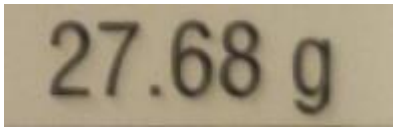
下面是完成这一切的代码：

```
def crop_each_bounding_box_and_ocr(自我):
自我。Table = []
Current_row = []
Image_number = 0
对于 self.rows 中的 row:
For bounding_box in row:
X, y, w, h = bounding_box
Y = Y - 5
Cropped_image = self。original_image [y: y + h x: x + w)
Image_slice_path = "。/ocr_slices/img_" + str(image_number) + ".jpg"
cv2。imwrite (image_slice_path cropped_image)
Results_from_ocr = self.get_result_from_tersseract（image_slice_path）
current_row.append (results_from_ocr)
Image_number += 1
self.table.append (current_row)
Current_row = []
Def get_result_from_tersseract(self, image_path):
Output = subprocess。getoutput('tesseract ' + image_path + ' -- l
eng———oem 3———psm 7———dpi 72 -c
tessedit_char_whitelist="ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789(). .cal
mg *")
Output = Output .strip（）
返回输出
```

正如你在上面看到的，我们正在使用 imwrite

我们将**所有的图像切片保存到一个文件**中，然后调用 Tesseract 命令行工具。

每个带有单个单词的图像切片看起来像这样：



以下是关于 Tesseract 命令行用法的两个**官方文档**：

1. TESSERACT(1) Manual Page
2. 命令行使用页面

关于使用 Tesseract CLI 命令的注意事项：

1. 注意 **tessedit_char_whitelist**，我用它来**约束图像中可能出现的字符**。这有助于提高准确性。
2. 我**通过-l eng 告诉该工具该语言是英语**
3. 我已经要求它使用一些最新的**基于神经网络的 OCR: -oem 3**

4. 我已经告诉它将图像视为**单行文本，使用：-psm 7**
5. 我**通过：-dpi 72 告诉它图像的 DPI 为 72**

为了得到最好的效果，你可能不得不玩弄这些值。上面的 2 个链接应该可以帮助你微调这一点。

当这个过程完成后，**你将有一个数组，每一行都有一个子数组。**现在，我们需要将其转换为 CSV。

步骤 C6：生成 CSV 文件

将其转换为 CSV 很简单。代码如下：

```
def generate_csv_file(自我):
用 open（"output.csv", "w"）作为 f：
对于 self.table 中的行：
F.write （",".join(row) + "\n"）
```

这**没什么可说的。**发生的事情是相当清楚的。代码运行后，您将得到一个名为 output.csv 的文件

进一步的改善

提高 OCR 的准确性

我发现的一件事是，**《Tesseract》**在将图像转换为文本方面并不像我希望的那样出色。它**在大多数情况下都有效，**但在某些情况下不起作用。您可以尝试将 **Tesseract 切换为** AWS text 或**谷歌云视觉 API**

《Tesseract》很棒，免费且**开源**（与上述选项不同），但与此同时，它很**老了，多年没有更新。**所以，你用其他服务可能会得到更好的结果。

使用启发式来提高结果的准确性

当您尝试时，您**可能会意识到您总是得到错误的 OCR 结果，但以一种可预测的方式。**例如，在营养标签的情况下，你**可能会发现你经常看到这样的东西：“FAR”而不是“FAT”**出于某种原因。在 OCR 过程的最后，你可以运行一些**通常转换的表格。**这些只是简单的文本替换操作，比如：

1. FAR- 100FAT
2. 吃->FAT
3. 12 - 12 克

你懂的。具体的规则将取决于问题领域和经常出现的错误类型。

结论

有了这些，我希望你对如何在 Python 中使用 OCR 从图像中提取表有一个清晰的想法。上面的过程需要根据你的用例进行调整。一些值和设置将不得不增加或减少。但是，如果你了解发生了什么，你应该能够导航所有这些变化。

发表于 AI， OpenCV, Python

其他相关帖子

使用 Python 构建自定义聊天机器人（自定义知识库）

想要取得联系？联系-at- livefiredev.com

关于：一组无废话的免费笔记，帮助开发人员帮助他人。

为什么这么空白？你想要杂乱吗？这是件艺术品：-)。这样的声明。